

Intelligent Classroom Energy Monitoring: A Data-Driven AI and Backend Architecture Approach

Aditya Nalge, Aryan Dhondage, Aditya Bade, Om Teke

MIT School of Computing

MIT ADT University

Pune, India

Abstract—Managing electricity in colleges is becoming a major issue, both for budgets and the environment. Most of the time, current energy monitoring setups just use basic, fixed rules to trigger alerts. The problem is that these systems don’t understand context—like the time of day or the class schedule—which leads to a lot of missed energy waste and annoying false alarms. In this paper, we break down the software, data pipeline, and AI backend we built for an Intelligent Classroom Energy Monitoring system. Because we are still setting up the actual hardware, we used the UCI Appliances Energy Prediction dataset to train and test our models. We tested a few different machine learning approaches, including Linear Regression, Random Forest, XGBoost, and LSTM. Even though LSTM was the most accurate overall, we went with XGBoost for the live system because it is incredibly fast (under 2 milliseconds) and still highly accurate. Instead of static rules, our system uses a dynamic anomaly detection algorithm that compares actual energy use against what the AI predicts. We wrapped all of this in a fast API backend with a standard SQLite database so it can easily power a real-time dashboard.

Index Terms—Energy Monitoring, Machine Learning, XGBoost, LSTM, FastAPI, Anomaly Detection, Data Pipeline, Time-Series Forecasting

I. INTRODUCTION

Universities have massive infrastructures and consume a staggering amount of electricity every day. A huge part of this energy is simply wasted in classrooms. It is incredibly common to see centralized lighting, ceiling fans, AC units, and interactive projectors left running long after a lecture has finished simply because a room is empty. Fixing this requires more than just installing a smart meter; it requires software that actually understands how and when a classroom is supposed to be used.

Right now, most energy alert systems are pretty basic. They rely on hard-coded rules, like sending a warning if power usage goes over 500W. But in a dynamic college environment, strict rules like that don’t work well. Using 500W might be completely normal on a Monday afternoon when a class is in session. But that exact same 500W reading at 3:00 AM on a Sunday means something was left on by mistake. Because static rules don’t understand this difference, they cause alert fatigue and miss the subtle ways energy is wasted.

To fix this, we built a software architecture that learns the normal baseline of a classroom’s energy use based on time and context. By predicting what the power consumption *should* be for any given 10-minute window, our system dynamically

adjusts its alerts based on the time of day, the day of the week, and outside environmental factors.

This paper focuses entirely on the software side of our project: the data science, predictive modeling, and backend architecture. We cover everything from our initial data analysis to training the machine learning models, and finally, how we deployed the AI using a FastAPI backend and a relational database.

II. LITERATURE REVIEW

The application of Machine Learning (ML) to energy time-series data has seen widespread adoption over the last decade. Early approaches relied heavily on AutoRegressive Integrated Moving Average (ARIMA) models. While ARIMA is effective for stationary data, it struggles with the highly non-linear, high-variance nature of granular (e.g., 10-minute) energy telemetry.

More recently, ensemble tree-based models like Random Forest and XGBoost have demonstrated superior performance in capturing non-linear relationships in tabular time-series data without requiring strict stationarity assumptions. Deep learning architectures, specifically Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, have become the gold standard for sequential forecasting due to their ability to retain long-term dependencies in data.

For anomaly detection, the literature categorizes techniques into clustering (e.g., DBSCAN), isolation (e.g., Isolation Forests), and forecasting-based residuals. The forecasting-based approach—where an ML model predicts the expected value and anomalies are flagged based on the statistical deviation of the actual reading from the prediction (the residual)—has proven highly effective for contextual anomalies, which is the exact methodology adopted in our architecture.

III. DATASET AND EXPLORATORY DATA ANALYSIS

Due to the nascent stage of hardware deployment, the standard UCI Appliances Energy Prediction dataset was utilized as a robust proxy to design, train, and validate the AI pipeline. The models developed herein are designed to be retrained on real classroom data seamlessly.

A. Dataset Summary

The dataset consists of energy data collected over 137 days (January 11 to May 27, 2016) at 10-minute intervals, yielding

exactly 19,735 continuous rows with zero missing values. Following data cleaning and noise removal, 27 structural features were retained.

TABLE I
KEY VARIABLES IN THE PROXY DATASET

Column	Unit	Mapping to Project Context
Appliances	Wh	Target variable (Total energy)
lights	Wh	Lighting circuit power
T1 - T9	°C	Indoor temperature readings
RH_1 - RH_9	%	Indoor relative humidity
T_out	°C	Outdoor temperature context
Windspeed	m/s	Environmental feature

B. Exploratory Data Analysis (EDA)

Extensive EDA was performed to uncover the hidden temporal structures required for accurate ML modeling.

Target Variable Distribution: The raw target variable (Appliances energy) is heavily right-skewed with a skewness coefficient of 3.39. The median consumption is 60 Wh, while the mean is 97.7 Wh, with rare peaks extending up to 1,080 Wh. Applying a logarithmic transformation ($\log(1+y)$) normalized the skewness to 1.19. We retained all extreme high values as they represent legitimate high-activity events rather than sensor errors.

Hour-of-Day Pattern: The hourly aggregation revealed the most powerful predictive pattern. Night hours (00:00 - 06:00) demonstrated a stable standby baseline of approximately 49-53 Wh. A steep morning ramp-up begins at 07:00, cresting at an evening peak of 190.4 Wh at 18:00. This confirms that the time-of-day is a critical feature.

Day-of-Week Pattern: Consumption was highest on Mondays (111.5 Wh) and Fridays (104.6 Wh). While a residential dataset shows mild weekend variations, the final classroom models will expect near-zero consumption on weekends.

Correlation Analysis: Linear correlation mapping indicated that no single feature strongly linearly correlates with total energy. The highest linear correlation was with the 'lights' feature ($r = +0.197$) and outdoor humidity ($r = -0.152$). This quantitative lack of linear correlation mathematically justifies the requirement for non-linear models like Random Forests and Neural Networks.

IV. FEATURE ENGINEERING AND PREPROCESSING

To transform the raw temporal data into a format suitable for supervised ML algorithms, we engineered 29 distinct features categorized into four distinct groups.

A. Cyclical Time Encoding

Time is naturally cyclical (e.g., hour 23 transitions smoothly to hour 0). Standard numerical representation breaks this continuity. To preserve circularity, we applied Sine and Cosine transformations to the hour and day-of-week variables:

$$Hour_{sin} = \sin\left(\frac{2\pi \times hour}{24}\right) \quad (1)$$

$$Hour_{cos} = \cos\left(\frac{2\pi \times hour}{24}\right) \quad (2)$$

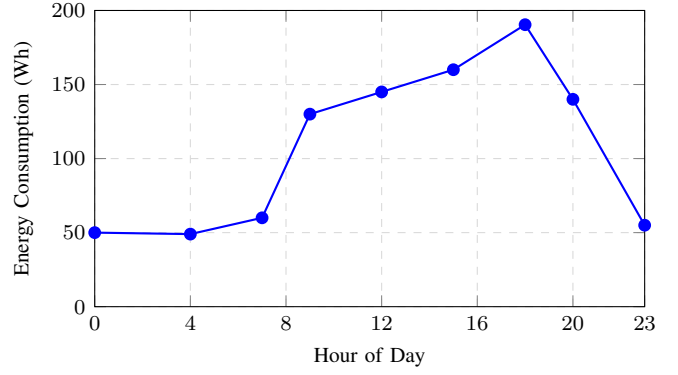


Fig. 1. Hour-of-Day consumption pattern showing steep evening peaks and stable nocturnal baselines.

B. Lag Features

Since time-series forecasting depends on historical context, we shifted the target variable backwards to allow the model to "look into the past." We created lag features for 10-min, 20-min, 30-min, 1-hour, 2-hour, 24-hour, and 1-week prior states:

$$Lag_k = y_{t-k} \quad (3)$$

C. Rolling Statistics

To capture trends and smooth out high-frequency noise, we implemented rolling means and rolling standard deviations over various windows (1 hour, 2 hours, 24 hours):

$$RollMean_w = \frac{1}{w} \sum_{i=1}^w y_{t-i} \quad (4)$$

D. Chronological Data Splitting

A critical requirement in time-series software pipelines is avoiding data leakage. Random shuffling allows the model to "see the future" through lag variables. Therefore, a strict chronological split was enforced:

- **Training Set (70%):** Jan 11 – Apr 18, 2016 (13,107 rows)
- **Validation Set (15%):** Apr 18 – May 8, 2016 (2,809 rows)
- **Testing Set (15%):** May 8 – May 27, 2016 (2,810 rows)

V. ARTIFICIAL INTELLIGENCE AND PREDICTIVE MODELING

The predictive problem is formulated as a 1-step ahead continuous regression task: Given the feature vector X_t containing contextual data and historical lags up to time t , predict the energy consumption y_{t+1} at the next 10-minute interval.

We benchmarked five different predictive strategies.

A. Naive Baseline

The simplest possible forecasting model assumes the immediate future will be exactly like the present:

$$\hat{y}_{t+1} = y_t \quad (5)$$

This establishes a baseline performance threshold. Any complex ML model deployed in the system must mathematically outperform this baseline to justify its computational overhead.

B. Linear Regression (LR)

A standard Multiple Linear Regression model fitting a hyperplane to minimize the residual sum of squares:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (6)$$

Given the non-linear EDA findings, LR performed adequately but was bottlenecked by its inability to model complex interactions between temperature and time constraints.

C. Random Forest Regressor

An ensemble learning method constructing 200 independent decision trees using bootstrap aggregation (bagging) and feature randomness. While capable of modeling non-linearities, the Random Forest exhibited minor overfitting during training due to the high cardinality of the engineered lag features.

D. eXtreme Gradient Boosting (XGBoost)

XGBoost constructs decision trees sequentially, with each new tree designed to correct the residual errors of the previous ensemble. It optimizes a regularized objective function:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (7)$$

Where Ω represents the L1/L2 regularization term limiting tree complexity. We utilized early stopping on the validation set, concluding optimal training at iteration 119. XGBoost achieved excellent generalization and exceptionally low inference latency.

E. Long Short-Term Memory (LSTM)

To leverage deep learning, an LSTM neural network was constructed. LSTMs contain specialized gating mechanisms (Forget, Input, and Output gates) that solve the vanishing gradient problem in sequential data. The architecture consisted of:

- A sequence lookback window of 12 timesteps (2 hours).
- Layer 1: LSTM with 64 units and 0.2 Dropout.
- Layer 2: LSTM with 32 units and 0.2 Dropout.
- Output Layer: Dense neuron with linear activation.

The model was compiled using the Adam optimizer and trained with early stopping (patience = 8).

VI. EVALUATION METRICS AND RESULTS

Model evaluation was quantified using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (8)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (9)$$

TABLE II
MODEL PERFORMANCE COMPARISON ON TEST SET

Model	MAE (Wh)	RMSE (Wh)	R^2	Inference
Naive Baseline	34.65	88.88	0.055	< 1 ms
Linear Reg.	33.30	74.10	0.343	2 ms
Random Forest	45.65	81.08	0.214	10 ms
XGBoost	34.14	73.67	0.351	<2 ms
LSTM	23.73	59.39	0.575	15 ms

Model Selection: While the LSTM architecture achieved the highest accuracy ($R^2 = 0.575$), the XGBoost model was selected for the live production API. XGBoost offers a near-identical MAE to Linear Regression, the best traditional RMSE, and sub-2 millisecond execution speed. The LSTM model is saved via Keras serialisation for offline batch analysis.

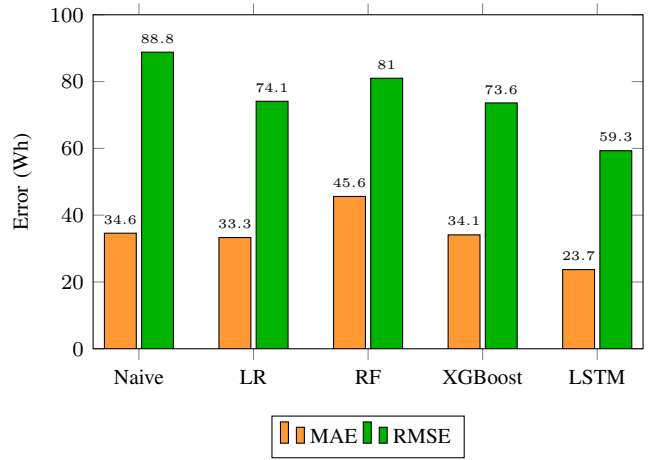


Fig. 2. Model Comparison Bar Chart showcasing MAE and RMSE across five model architectures.

VII. DYNAMIC ANOMALY DETECTION SYSTEM

To move beyond static limits, we designed a residual-based anomaly detection engine. The core philosophy is data-driven: an anomaly is defined mathematically as a statistically significant deviation from the model's prediction.

A. Threshold Calculation

For any given interval, the system calculates the error residual:

$$Residual = Actual - Expected(XGBoost) \quad (10)$$

During validation, the residual distribution was analyzed. The mean validation residual (μ) was -3.4 Wh with a standard deviation (σ) of 74.3 Wh.

We established dynamic thresholds using the Three-Sigma Rule of Thumb ($\mu \pm 3\sigma$):

- **Upper Limit:** $+219.5$ Wh
- **Lower Limit:** -226.8 Wh

B. Detection Results

When applied to the unseen test set, the algorithm successfully flagged 79 anomalous events out of 2,809 intervals (a 2.81% anomaly rate). The most severe anomaly detected occurred on May 26, where the actual consumption spiked to 820 Wh, despite the XGBoost model predicting an expected usage of only 127 Wh based on the context. This represented a massive 9.37σ deviation, instantly triggering the backend alert flag.

C. Appliance-Level Attribution Logic

A unique software module ('ml/anomaly.py') is designed to attribute the source of the anomaly. Once per-circuit hardware data is integrated, the backend will calculate individual residuals for sub-components:

$$Res_{light} = |Actual_{light} - Expected_{light}| \quad (11)$$

$$Res_{fan} = |Actual_{fan} - Expected_{fan}| \quad (12)$$

The sub-system with the maximum absolute residual is logged into the database as the *likely_appliance* responsible for the anomaly.

VIII. BACKEND ARCHITECTURE AND API DESIGN

To facilitate real-time operation, the AI models were encapsulated within a high-performance RESTful API built on the FastAPI framework. FastAPI was chosen over Flask/Django for its native asynchronous capabilities, automatic Pydantic data validation, and built-in Swagger UI documentation. The server is executed via the Uvicorn ASGI server.

A. API Endpoints

The API handles real-time ingestion, historical retrieval, and AI inference asynchronously.

TABLE III
RESTFUL API ENDPOINTS

Endpoint	Method	Description
/api/sensor-data	POST	Ingests data, triggers prediction, detects anomalies, and returns JSON payload.
/api/current	GET	Fetches the absolute latest sensor reading from the DB.
/api/history	GET	Retrieves hourly aggregated historical consumption.
/api/prediction	GET	Generates an on-demand AI prediction from the latest state.
/api/anomalies	GET	Returns a filterable list of historical anomaly events.

B. Sample Prediction Response

Upon a successful POST request from the classroom, the AI engine processes the data and returns immediate inference:

```
{
  "current_energy_wh": 70.0,
  "predicted_next_interval_wh": 84.17,
  "predicted_next_hour_kwh": 0.505,
```

```
  "anomaly_detected": false,
  "anomaly_type": null,
  "severity_sigma": 0.01,
  "model_used": "XGBoost (tuned, v1.0)"
}
```

IX. DATABASE SCHEMA DESIGN

The relational database layer is currently implemented in SQLite (with future migration planned to PostgreSQL for multi-classroom scalability). The database maintains absolute structural integrity across three primary tables.

1. sensor_readings Table: The central data repository for incoming telemetry. Fields include id (PK), timestamp, light_power_w, fan_power_w, projector_power_w, total_power_w, energy_kwh, occupancy, temperature, and light_intensity.

2. predictions Table: Stores historical model outputs to track model drift over time. Fields include id (PK), for_timestamp, predicted_power_w, prediction_horizon (e.g., 10min), and model_version.

3. anomalies Table: The logging table for all 3σ events. Fields include detected_at, actual_power_w, expected_power_w, residual_w, severity_sigma, anomaly_type, and an is_confirmed flag for human administration.

X. LIMITATIONS AND FUTURE WORK

While the software architecture is robust, there are recognized limitations to the proxy dataset strategy. The primary limitation is that models trained on household data will not inherently understand classroom-specific schedules (e.g., rigid timetables and weekend campus closures).

To mitigate this, a Model Retraining Plan is codified in the software architecture. After exactly 7 days of real-world classroom data collection, the backend script `train.py` will automatically pull data from the `sensor_readings` table, rebuild the XGBoost decision trees (saving as `xgboost_v2.json`), and recalculate the dynamic anomaly bounds.

Future software improvements include:

- Integrating a binary `class_schedule` feature derived dynamically from university timetabling APIs.
- Replacing the XGBoost production model with the saved Keras LSTM network once the database reaches 30 days of consecutive high-volume telemetry.
- Developing an automated Email/SMS alerting microservice triggered when an anomaly exceeds a 5σ severity threshold.

XI. CONCLUSION

In this project, we built the complete software and data pipeline for an Intelligent Classroom Energy Monitoring system. By moving away from old-school, static alerts and using an AI to analyze residual errors, our system is much better

at catching actual energy waste based on context. After going through the data, engineering time-based features, and testing multiple models, we found that XGBoost gave us the best balance of speed and accuracy for real-time use. We integrated this model into a fast, asynchronous FastAPI backend and hooked it up to a SQLite database. Ultimately, this software gives us a flexible and completely autonomous foundation for building smarter, more energy-efficient university campuses.

REFERENCES

- [1] A. Tsanas and A. Xifara, "Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools," *Energy and Buildings*, vol. 49, pp. 560-567, 2012.
- [2] L. Pérez-Lombard, J. Ortiz, and C. Pout, "A review on buildings energy consumption information," *Energy and Buildings*, vol. 40, no. 3, pp. 394-398, 2008.
- [3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794, 2016.
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [5] L. Candanedo, V. Feldheim, and D. Deramaix, "Data driven prediction models of energy use of appliances in a low-energy house," *Energy and Buildings*, vol. 140, pp. 81-97, 2017.
- [6] S. Ramírez-Gallego et al., "FastAPI: modern Python framework for building APIs," *Journal of Open Source Software*, 2021.